



Semaine 2 : Comprendre et manipuler les branches dans Git



Durée : 3h



Objectifs :

- Comprendre ce qu'est une branche et pourquoi elle est essentielle en développement.
- Apprendre à créer, naviguer et gérer des branches.
- Fusionner des branches et gérer les conflits.
- Découvrir le fonctionnement de GitHub pour collaborer sur un projet.



1 Introduction aux branches

Théorie

Dans Git, une **branche** représente une ligne de développement indépendante. Elle permet de travailler sur des fonctionnalités ou corrections **sans impacter la branche principale (main)**.



Pourquoi utiliser des branches ?

- Expérimenter sans risquer de casser le projet.
- Travailler sur plusieurs fonctionnalités en parallèle.
- Permettre une collaboration fluide en équipe.



Types de branches courants :

- `main` → Branche principale stable.
- `feature/*` → Branche pour une nouvelle fonctionnalité.
- `bugfix/*` → Branche pour corriger un bug.
- `hotfix/*` → Branche pour une correction urgente en production.

2 Manipulation des branches

Théorie

Créer et basculer entre les branches

Chaque branche dans Git est comme un instantané du projet à un moment donné.

Démonstration

```
# Vérifier les branches existantes
git branch

# Créer une nouvelle branche (sans basculer dessus)
git branch feature/nouvelle-fonctionnalité

# Basculer sur cette branche
git checkout feature/nouvelle-fonctionnalité

# Nouvelle syntaxe recommandée (Git 2.23+)
git switch feature/nouvelle-fonctionnalité

# Supprimer une branche locale (après fusion)
git branch -d feature/nouvelle-fonctionnalité
```

Exercice 1 : Création et navigation entre branches

1. **Créer une branche feature/test.**
2. **Basculer sur cette branche et modifier un fichier** (README.md par exemple).
3. **Faire un commit et revenir sur main.**
4. **Constater que les modifications ne sont pas sur main.**

3 Fusionner des branches et gérer les conflits (1h)

Théorie

Une **fusion (merge)** consiste à intégrer les modifications d'une branche dans une autre.
Deux types de fusion :

1. **Fast-forward** (si main n'a pas bougé, Git avance simplement le pointeur).

2. **Merge commit** (si main a reçu des changements entre-temps).

💡 Pourquoi des conflits surviennent-ils ?

Un conflit arrive lorsque **deux branches modifient la même ligne** dans un fichier. Git ne peut pas choisir quoi garder.

Démonstration

```
# Revenir à la branche principale  
git checkout main
```

```
# Fusionner la branche feature/nouvelle-fonctionnalité  
git merge feature/nouvelle-fonctionnalité
```

⚠️ **Cas de conflit**

```
Auto-merging fichier.txt  
CONFLICT (content): Merge conflict in fichier.txt
```

💡 Comment résoudre un conflit ?

1. **Ouvrir le fichier en conflit** et choisir quelle version garder.
2. **Supprimer les marqueurs de conflit** (<<<<<<, =====, >>>>>>).
3. **Ajouter le fichier résolu avec** `git add`.
4. **Finaliser la fusion avec** `git commit`.

Exercice 2 : Simuler et résoudre un conflit

1. **Créer une branche** `conflict-test` et modifier une ligne dans `test.txt`.
2. **Faire un commit**, puis revenir sur `main` et **modifier la même ligne**.
3. **Tenter une fusion** et constater le conflit.
4. **Résoudre le conflit** et valider.

Introduction à GitHub et collaboration (30 min)

Théorie

GitHub est une plateforme qui permet :

- D'héberger des projets Git.
- De travailler à plusieurs sur un même code via des **pull requests**.
- De suivre l'historique des changements et proposer des améliorations.

Démonstration

Créer un dépôt distant et y pousser du code

```
# Lier le dépôt local à un dépôt GitHub  
git remote add origin https://github.com/utilisateur/mon-projet.git
```

```
# Pousser le projet sur GitHub  
git push -u origin main
```

💡 **git pull** permet de récupérer les dernières modifications

```
git pull origin main
```

Exercice 3 : Publier un projet sur GitHub

1. Créer un dépôt sur GitHub.
2. Le lier à son dépôt local.
3. Envoyer du code avec **git push**.
4. Tester **git pull** pour récupérer les modifications.